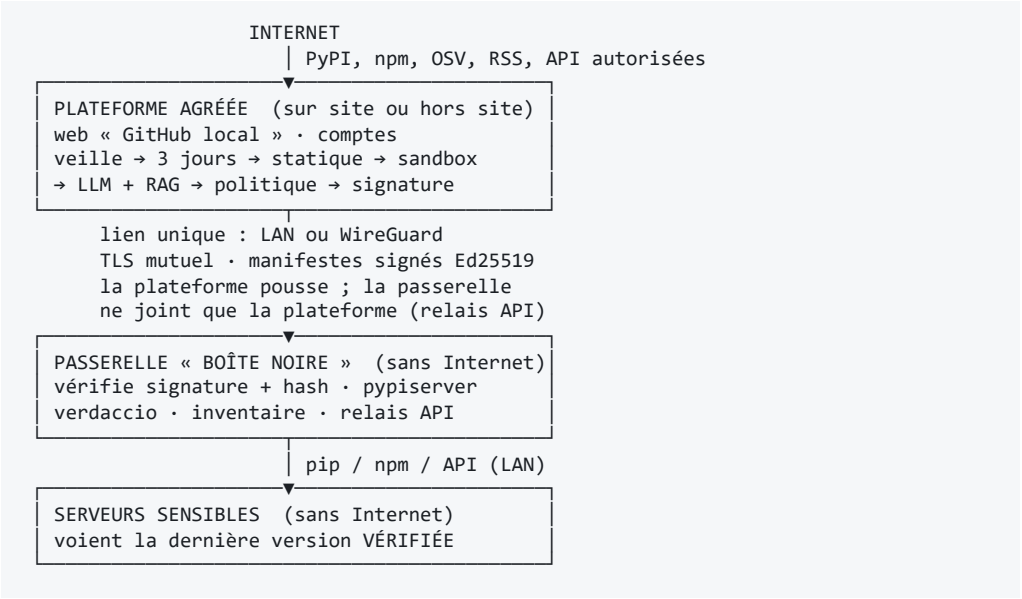
sas-pme — Roadmap du MVP

Hackathon Track 3 · A = Sam (plateforme web) · B = Mouabd19 (pipeline + IA) · C = STEUSSO (réseau + passerelle) · H0 = début de la roadmap · ~30 h

> **Vos serveurs n'installent que ce qu'on a vérifié.** Les serveurs sensibles de la PME n'ont pas Internet : ils prennent pip et npm dans une **passerelle sans Internet**, qui ne reçoit que des versions **attendues 3 jours, analysées localement** (code, sandbox, LLM + RAG) et **signées** par une **plateforme agréée**, la seule machine connectée. Objectif : **un MVP démontrable**, pas plus.



Principes non négociables

Les serveurs ne parlent jamais à Internet	tout passe par la passerelle, qui ne parle qu'à la plateforme
Le code décide, l'IA alerte	le LLM ou un article peut monter un risque, jamais approuver
Un texte lu n'est pas un ordre	code, changelog, article, RAG : des données ; sortie du LLM validée par schéma
Rien sans signature	la passerelle refuse un manifeste mal signé, un hash différent, un nonce rejoué
Tout local	LLM, RAG, sandbox sur la plateforme ; aucun service d'IA cloud
Journal inaltérable	append-only, chaîné par hash

Démo AWS (eu-west-3)

Sous-réseau public	plateforme · g4dn.xlarge (GPU T4) · Ollama qwen2.5-coder:7b + nomic-embed-text · Chroma · Docker
Sous-réseau **privé sans route Internet**	passerelle (t3.small) + serveur sensible (t3.small) : `curl pypi.org` échoue, `pip install` marche
Repli	quota GPU à 0 : c7i.4xlarge + modèle 3b

Planning — jalons et points de rencontre

Heure	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
Jalons	**M0 socle**			**M1 MVP pip**							**M2 différence**						**M3 finition**						**marge · sommeil · répétitions**							
Rencontres		**H1**		**H3**							**H10**						**H16**						**H22**							
A · Sam plateforme web	commun/ · db · web vide			comptes · catalogue · fiche version · file d'attente · demandes · journal							décision humaine · révocation · tableau de bord · inventaire						deck · fiche de défense · pitch													
																							marge · sommeil (1er) · répétition du pitch							
B · Mouabd19 pipeline + IA	client PyPI · demander() · politique factice			délai 3 j · veille OSV · statique + diff · LLM confiné · politique							RAG (Chroma) · sandbox dans le prompt · veille RSS · npm (ext.)						mesure finale (3 colonnes) + diapo													
																							bugs · sommeil (2e) · questions du jury sur la mesure							
C · STEUSSO réseau + passerelle	VPC · sous-réseau privé · pypiserver · stubs			récepteur · TLS mutuel · signature Ed25519 · inventaire · sandbox v1							sandbox v2 · relais API · sauvegardes · Verdaccio, WireGuard (ext.)						vidéo · schéma réseau · diapo « plateforme compromise ? »													
																							marge · sommeil (3e) · démo en direct prête							

Une colonne = une heure. Orange = point de rencontre (15 min, les trois, devant la démo de bout en bout). H1 = contrats `commun/` sur `main`. « ext. » = extension, coupée en premier.

Points de rencontre

Heure	Doit marcher de bout en bout	Sinon
H1	`commun/modeles.py` + `pyproject.toml` sur `main` ; quota GPU demandé	A ne fait rien d'autre avant
H3	demande de `requests` dans le web → téléchargée → approuvée (factice) → copiée sur la passerelle → `pip install` marche sur le serveur, `curl pypi.org` échoue	les trois sur le maillon qui casse
H10	**MVP** : version propre attendue, analysée, signée, publiée, installée ; version piégée bloquée avec preuves statiques + sandbox + phrase ; publication forgée refusée	c'est la démo minimale garantie : on la stabilise avant M2
H16	démo complète répétée une fois ; on gèle les fonctions	appliquer l'ordre de coupe
H22	vidéo, deck, mesure terminés	la vidéo passe avant le deck

> **Ordre de coupe :** 1. npm · 2. WireGuard (le LAN suffit en démo) · 3. veille RSS · 4. RAG · 5. sandbox v2. **Jamais coupés :** cloisonnement, signature, sandbox v1, mesure.

Tâches par jalon et par personne

Jalon	A Sam — plateforme web	B Mouabd19 — pipeline + IA	C STEUSSO — réseau + passerelle
M0 · H0–H3 **socle commun**	`commun/modeles.py` (contrats) **avant H1** ; `pyproject.toml` ; `plateforme/db` (schéma SQLite + `acces.py`) ; web : connexion fixe, catalogue vide	client PyPI (métadonnées, téléchargement) ; `pipeline.demander()` ; politique factice « tout approuver »	VPC + sous-réseau public + **sous-réseau privé sans Internet** ; pypiserver sur la passerelle ; stubs `sandbox.executer` et `signature.publier` (copie par scp)
En attendant	maquette des pages	corpus : 30 malveillants OpenSSF, 50 sains (ne jamais exécuter)	quota GPU, Ollama + modèles
M1 · H3–H10 **MVP pip**	comptes + rôles + sessions ; catalogue ; **fiche version** (verdict, signaux, preuves, rapport sandbox) ; **file d'attente** avec compte à rebours ; « demander un paquet » ; journal chaîné	**délai 3 j** (réglable en démo) ; veille OSV + statut PyPI ; **statique** : typosquat, âge, mainteneurs, OpenSSF, diff (code à l'installation, exec/base64, secrets, domaines) ; **LLM** confiné (JSON, phrase FR) ; **politique**	**récepteur** `/v1/publish`, `/v1/revoke`, `/v1/inventory` ; **TLS mutuel** ; **Ed25519** : signer (plateforme), vérifier hash + âge + nonce (passerelle) ; inventaire ; **sandbox v1** (sans réseau, faux DNS, leurres)
En attendant	script de démo	mesure v1 (statique seul)	durcissement passerelle (ufw, utilisateur sans droits)
M2 · H10–H16 **différence**	décision humaine ; **révocation** ; tableau de bord ; page inventaire ; (ext.) affichage npm	**RAG** : OSV/GHSA, OpenSSF, CWE, CAPEC, ATT&CK → Chroma, références citées ; rapport sandbox dans le prompt ; veille RSS ; (ext.) contrôles npm	**sandbox v2** (strace, battement 60 s) ; **relais API** en liste fermée ; **sauvegardes** restic + test de restauration ; (ext.) Verdaccio, WireGuard
M3 · H16–H22 **finition**	deck 8 diapos ; fiche de défense ; répétition	**mesure finale** : détection / fausses alertes en 3 colonnes (statique ; + sandbox ; + LLM/RAG) + diapo	**vidéo** ≤ 3 min ; schéma réseau ; diapo « et si la plateforme est compromise ? »

La démo (3 minutes)

1. serveur : `curl pypi.org` échoue, `pip install requests` marche — *le serveur ne parle jamais à Internet*
2. web : la nouvelle version attend ses 3 jours, puis est approuvée, signée, publiée ; `pip install -U` la reçoit
3. version piégée bloquée : la sandbox l'a vue appeler son serveur et lire un faux jeton ; le LLM l'explique en citant un cas du RAG
4. publication forgée refusée par la passerelle — *même une plateforme usurpée ne publie rien*
5. diapositive de mesure

Risques

Quota GPU à 0	demande dès H0 ; repli CPU + 3b
« Où est l'IA ? »	mesure en 3 colonnes ; sandbox + RAG cités dans le verdict
Docker dans la sandbox	v1 simple dès M1 ; v2 seulement si v1 tient
Injection de prompt	le LLM ne peut que monter un risque ; test dédié
Échantillons malveillants	jamais exécutés hors sandbox ; Python en `-I`

Comment on travaille, et les sauvegardes

Branches : pourquoi pas trois branches longues

Trois branches longues divergent en quelques heures : chaque fusion devient une crise, et chacun code contre une version périmée du travail des autres — c'est « travailler dans le vide ».

> **Retenu :** `main` comme tronc + **une branche courte par tâche** (`a/…`, `b/…`, `c/…`), vivante 1 à 3 h, fusionnée par PR dès que les tests passent. Chacun repart de `main` à chaque tâche et voit le travail des autres en quelques heures. Pas de conflit : chacun possède ses dossiers.

Qui possède quoi

A	`plateforme/web/`, `plateforme/db/`, `demo/script.md`
B	`plateforme/pipeline/`, `mesure/`, `demo/paquets/`, `demo/corpus/`
C	`plateforme/sandbox/`, `plateforme/signature/`, `passerelle/`, `infra/`, `demo/reseau.md`
Commun	`commun/` (contrats) : **PR relue par les trois** ; `docs/` ; `pyproject.toml` (une section par personne)

Stubs

Chaque interface de `docs/contrats.md` existe dès M0 en faux : B code le pipeline avant la sandbox de C, A affiche une fiche avant le verdict réel de B. Le vrai remplace le faux sans rien changer chez les autres.

Agents (Claude Code, Kiro)

`CLAUDE.md` et `.kiro/steering/sas-pme.md` font lire `docs/` à chaque prompt. L'agent demande « A, B ou C ? » puis reste dans les dossiers de cette personne.

Sauvegardes — le local les rend plus faciles

Tout ce qui compte vit sur deux machines connues ; presque tout est reconstructible (les paquets sont publics). Ce qui ne l'est pas : **notre jugement** — verdicts, journal, clés.

Donnée	Perte	Reconstructible
Clé privée de signature	critique	non
Base : verdicts, journal, comptes	forte	non
Archives approuvées	moyenne	oui (re-téléchargement + hash)
Dépôts de la passerelle	faible	oui (republication)
Index RAG, modèles	faible	oui

Schéma 3-2-1

1. **Locale quotidienne** : restic chiffré vers un NAS ou disque sur site, 30 jours
2. **Hors ligne hebdomadaire** : disque USB chiffré débranché (contre un rançongiciel)
3. **Cloud : oui, en option, chiffré côté client** (restic → S3 ou hébergeur français), clé détenue par la PME seule ; seule copie qui survit à un incendie

> **La clé privée de signature ne va jamais dans le cloud** : deux copies hors ligne (USB chiffré au coffre, papier).

Dans le MVP (M2, C) : `infra/sauvegarde.sh` + test de restauration (effacer la base, restaurer, vérifier la chaîne du journal). Le cloud : feuille de route.

Après le hackathon

Priorité n° 1 : la plateforme d'intégration — installation en une commande, configuration automatique de pip/npm sur les serveurs, image d'appliance : le « seamless » chez une PME. Puis npm complet, Docker/Maven/NuGet, console multi-clients pour infogérants, télémétrie partagée.